Use Cases and Traceability

UC-01: Discover a temple

Actor: Guest or authenticated user
Precondition: API and database are available.
Flow: Open home → optionally enter name/location → submit search → select a temple → inspect details, events, ratings, amenities, and slots.
Outcome: Matching temple and available future slot information is shown.
Code: `Home.jsx`, `TempleDetails.jsx`, `templeController.js`, `slotController.js`.

UC-02: Register and sign in

Actor: Guest
Flow: Enter identity details → solve client-side math captcha → submit → backend hashes password → backend returns JWT → client stores token in `localStorage`.
Outcome: Profile is loaded and the user can call protected endpoints.
Security gap: The client and API permit self-selection of `ORGANIZER` or `ADMIN`. The captcha is client-only.

UC-03: Book darshan

Actor: USER
Precondition: Valid token and sufficient seat capacity.
Flow: Choose slot → enter 1–10 devotees in the UI → request hold → backend deducts seats and creates `Pending` booking → five-minute timer begins → enter simulated card data → confirm → status becomes `Booked` → print ticket.
Alternates: Insufficient capacity returns 400; missing slot returns 404; expired hold returns 400; repeat confirmation returns 400.
Code: `BookingModal.jsx`, `bookingController.js`, `Booking.js`, `DarshanSlot.js`.

UC-04: Cancel a booking

Actor: Owner, organizer, or admin
Flow: Call cancellation endpoint → server checks ownership for a `USER` → booked seats are restored → booking becomes `Cancelled`.
Known defect: Pending holds changed directly to `Cancelled` do not restore capacity. The current user dashboard defines a cancellation handler but does not expose a cancellation button in the booking list.

UC-05: Make a donation

Actor: Authenticated user
Flow: Select temple, amount, purpose, and donor visibility → enter simulated card fields → client validates format → backend records donation and generates `TXN-########` → print receipt.
Note: No gateway authorization, capture, refund, or webhook exists.

UC-06: Organize slots

Actor: ORGANIZER or ADMIN
Flow: Select any temple → enter date/time/capacity/price → create slot; list or delete slots; inspect booking registry.
Known design gap: No temple ownership is stored on the organizer, so authorization is global.

UC-07: Administer the platform

Actor: ADMIN
Flow: View analytics; list/filter/delete users; create/delete temples; add events; log maintenance; advance maintenance status.
API-only functions: update a user, update a temple, delete an event.

Acceptance traceability matrix

Requirement	Primary API	UI	Acceptance test
FR-01/02	`POST /auth/register`, `POST /auth/login`	Register/Login	Valid account returns JWT; wrong password returns 401
FR-04/05	`GET /temples`, `GET /temples/:id`	Home/Temple Details	Partial filters return matching records
FR-06	`POST /temples/:id/reviews`	Temple Details	Second review updates same user's review
FR-07	`GET /slots`	Temple Details	Temple/date filters restrict response
FR-08–11	Booking endpoints	Booking Modal	Hold decrements once; confirm changes status; expiry restores once
FR-12/13	My bookings/cancel	User Dashboard	Only owner can cancel; all cancellable holds restore seats
FR-14/15	Donation endpoints	Donation/User Dashboard	Valid amount records receipt under user
FR-16/17	Slot/registry endpoints	Organizer Dashboard	Organizer sees only assigned temple data (not currently met)
FR-18–22	Temple/admin endpoints	Admin Dashboard	Non-admin gets 403; admin operations persist